

GRADUATION THESIS DEFENSE

毕业设计答辩

基于Spring Cloud+Vue3的预约挂号系统的设计与实现

答辩人：黄俊棋 | 指导老师：陈代进

2026年3月



目录



01. 研究背景与意义



02. 技术选型与架构



03. 系统设计与实现



04. 系统测试与结果



05. 论文总结与展望

1 研究背景：医疗系统现状痛点分析



门诊压力大，资源利用率低

三甲医院日均门诊量超5000人次，患者排队时间长，医疗资源利用率仅60%。



线上服务渗透率不足

2023年三级医院线上挂号率仅45.2%，远低于国家“互联网+医疗健康”80%的目标。



用户体验不佳

部分平台交互冗余、不直观，尤其在人口老龄化趋势下，不利于老年患者快捷操作。

核心痛点总结：医疗资源分配不均、数字化转型滞后、服务适老化不足，需智能化升级方案。

1 研究意义：项目建设意义



缩短患者挂号等待时间

实现全流程线上预约挂号，减少现场排队拥堵，优化就医体验。



提升医院运营效率

优化医疗资源配置，通过数据化管理合理调度人力与物力资源。



推动医疗数字化转型

助力传统医疗服务向数字化、智能化升级，构建智慧医疗新生态。



传统就医模式下的排队痛点，需数字化手段解决

2 技术选型与架构

整体架构：前后端分离架构 + 微服务治理体系，保障系统高可用与扩展性



后端核心技术

- Java JDK17 / Spring Cloud
- Nacos 服务注册与发现
- Gateway 统一网关路由
- Sentinel进行流量管制



数据存储

- MySQL 8.0 核心业务存储
- Redis 缓存加速与防超卖
- Navicat 可视化管理工具



前端交互体系

- Vue 3 + Element Plus
- 响应式适配 PC/移动端
- 组件化开发，高效迭代

2 技术选型与架构：开发平台——若依框架



前端框架：RuoYi-Cloud-Vue3

（若依微服务版前端，基于 Vue3 + Element Plus 构建）

复用若依现成能力（用户认证、权限控制、菜单路由、日志审计、代码规范），在此基础上按医院业务做功能扩展。

框架内部集成了基础的配置类以及基础交互界面，预留了各项中间件的配置，修改端口即可使用，同时提升开发效率

1、微服务架构

将系统拆分为独立服务（如用户服务、挂号服务、支付服务等），便于后续扩展和维护。

2、前后端分离

前端 Vue3 负责页面交互，后端微服务提供 API 接口，职责清晰，符合现代 Web 开发规范。

3、企业级组件集成

内置 Nacos 服务注册发现、Sentinel 流量控制等组件，可直接支撑挂号系统的高并发、分布式场景。

4、权限与安全体系

继承若依框架的 RBAC 权限模型，符合所开发的医疗系统的安全合规需求。



后端框架：RuoYi-Cloud

（若依微服务版，基于 Spring Cloud Alibaba 生态）

2 技术选型与架构：开发工具



后端开发：IDEA

提供高效的代码编写、调试和运行环境



前端开发：IDEA + WindSurf

支持前端代码开发，并借助大模型开发/联调提升开发效率



构建工具：Maven

项目构建、版本号控制



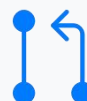
中间件：VMware + Docker

在虚拟机环境中部署 Redis，并通过 Docker 拉取与运行中间件镜像，构建隔离、可复现的开发测试环境。



版本控制：Git

版本管理



代码管理：GitHub

作代码远程仓库、代码托管、版本追踪



测试工具：JMeter

进行性能测试、压力测试和接口测试



图形设计：Visio

绘制系统架构图、流程图和数据模型图



总结：通过标准化开发工具链，保障了系统开发效率与版本稳定性。

2 技术选型与架构：安全保障体系



JWT + Redis会话

以JWT + Redis会话组合进行登录态存储，支持续期与可控失效，兼顾性能和安全性



流量治理（限流 + 熔断降级）

使用Sentinel中间件和降级工厂配置进行流量治理，避免请求堆积及故障扩散（FallbackFactory+Sentinel）



密码单向哈希加密

每次加密带随机盐，解密形式为哈希比对
比 MD5 更安全，更好暴力破解

2 技术选型与架构：逻辑架构图

 用户层：浏览器访问

 前端层 (Vue3)：患者端门户 / 医生工作台 / 管理后台

 网关层 (Gateway)：统一入口，负责路由转发、身份鉴权、请求限流

 认证服务 (Auth)：登录/权限

 系统服务 (System)：菜单/配置

 业务服务：科室/排班/预约/病历

 Nacos：服务注册与发现、配置中心 | Sentinel：全链路流量控制、熔断降级

 数据层：MySQL (核心业务数据) / Redis (热点缓存/会话)

3 系统设计与实现：系统架构设计



表现层

患者端 / 医生工作台 / 管理后台

技术栈: Vue 3.5.16 | Vite 6.3.5 | Element Plus 2.10.7



网关层

Spring Cloud Gateway (请求路由 / 接口聚合 / 统一认证)

功能: 流量控制(Sentinel) | CORS跨域处理 | 鉴权验证



微服务层

用户服务

注册/登录/账户管理

预约服务

预约/查询/号源管理

排班服务

班次/号源/冲突检测

系统管理服务

科室管理服务

医生信息服务

病历记录服务

患者信息服务

公共服务

认证服务

系统服务



数据与注册中心

MySQL 8.0

业务表/软删除/自动审计

Redis 6.0

Token缓存/分布式锁/队列

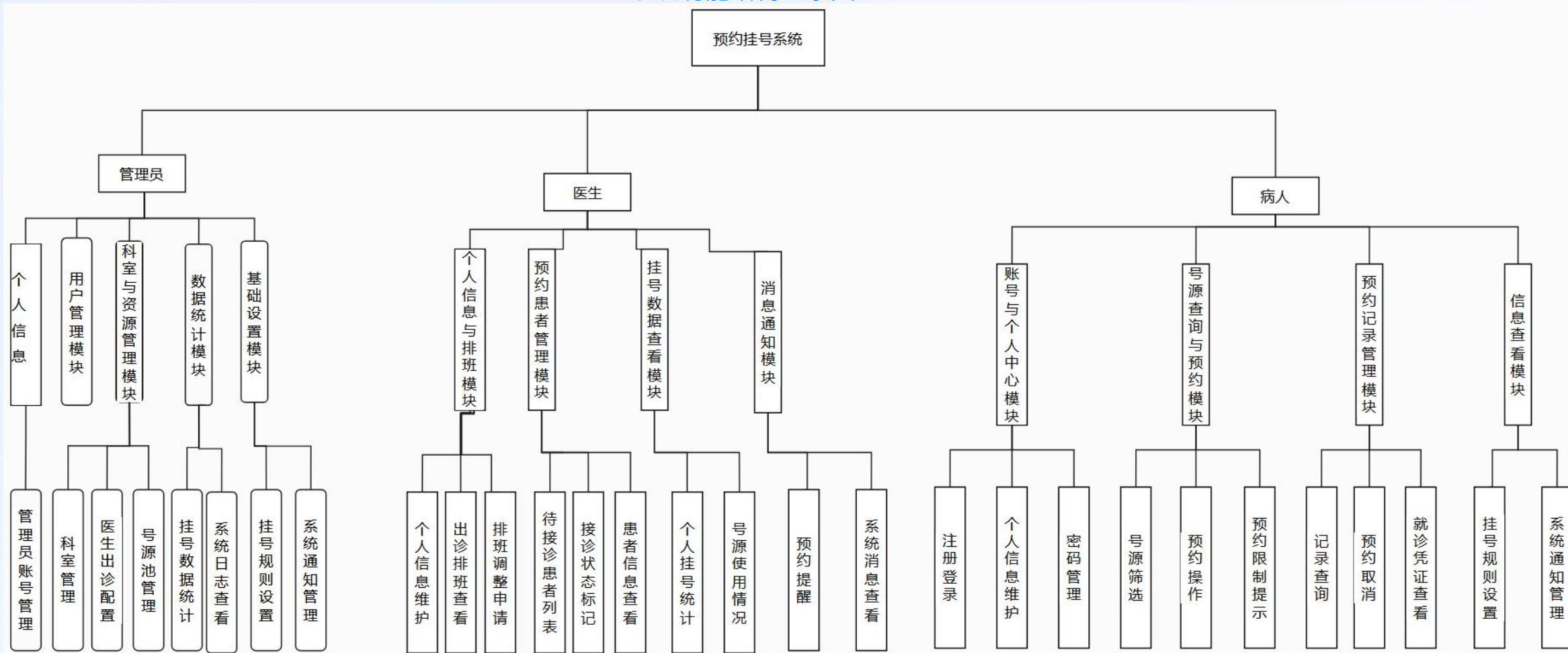
Nacos 2.0

服务注册/配置管理/健康检测

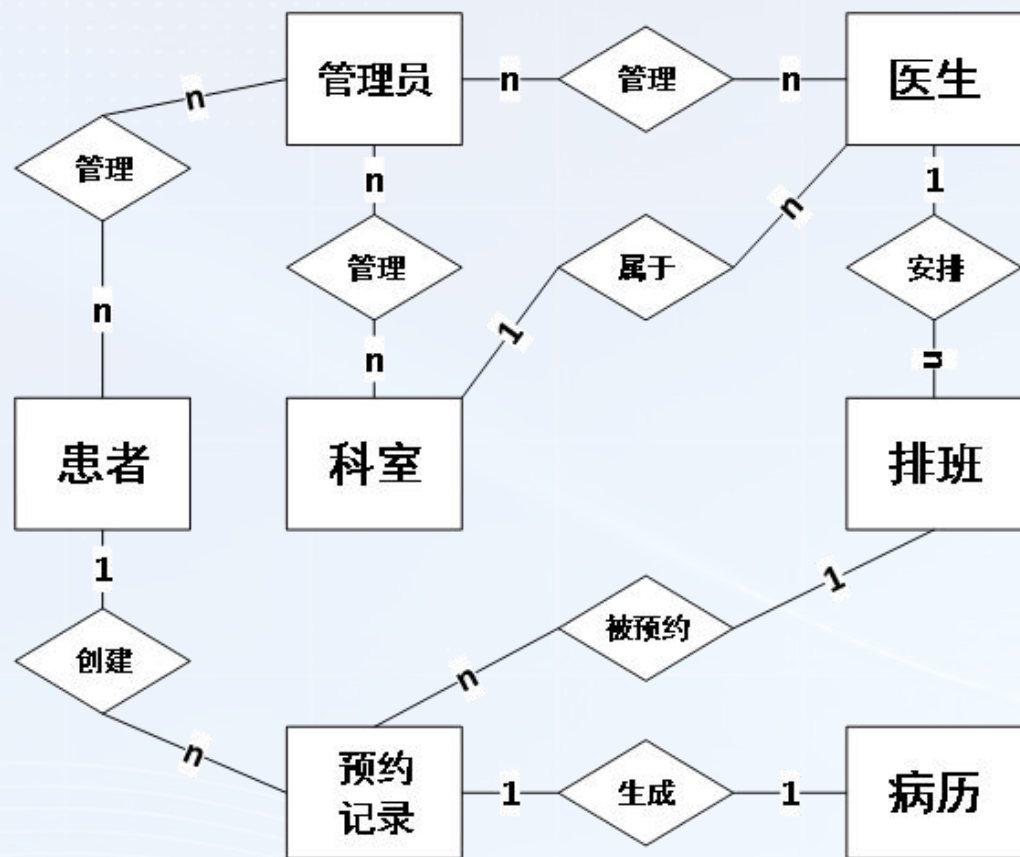
3 系统设计与实现：系统功能设计

系统功能按管理员、患者、医生三类角色划分，功能相互关联：患者可挂号预约，医生可就诊排班，管理员负责基础数据管理与操作审核。

系统功能结构全景图



3 系统设计与实现：数据库E-R模型与核心表设计



总E-R模型

核心数据表设计

1. 管理员表

管理员ID、登录账号、密码、手机号、管理员姓名、是否启用、创建时间、是否删除、删除时间、管理员等级

2. 科室表

科室ID、科室名称、创建时间、删除时间、是否删除

3. 医生表

医生ID、所属科室ID、登录账号、密码、医生姓名、职称、是否在职、创建时间、删除时间、是否删除

4. 患者表

患者ID、登录账号、密码、姓名、手机号、身份证号、创建时间、是否启用、删除时间、是否删除

5. 预约记录表

预约ID、患者ID、排班ID、就诊状态、预约时间、创建时间、是否删除、预约时段、删除时间

6. 病历表

病历ID、预约ID、患者ID、医生ID、诊断结果、处方信息、医嘱备注、就诊时间、创建时间、删除时间、是否删除

7. 排班表

排班ID、医生ID、出诊日期、班次、总号源数、剩余号源、创建时间、删除时间、是否删除

核心的数据表设计思路是围绕预约、挂号、排班三大核心功能设计，表间通过主外键关联，关键数据均采用逻辑删除，确保数据可溯源。

3 系统设计与实现：角色登录优化方案



优化思路

优化了系统登录页，删去角色选择，默认展示患者登录界面，医生与管理员通过专属路径登录。



操作流程

- 患者：无需选择角色，输入账号密码即可直接登录。
- 医生/管理员：需手动输入特定路由（/doc 或 /admin）访问后台登录页。



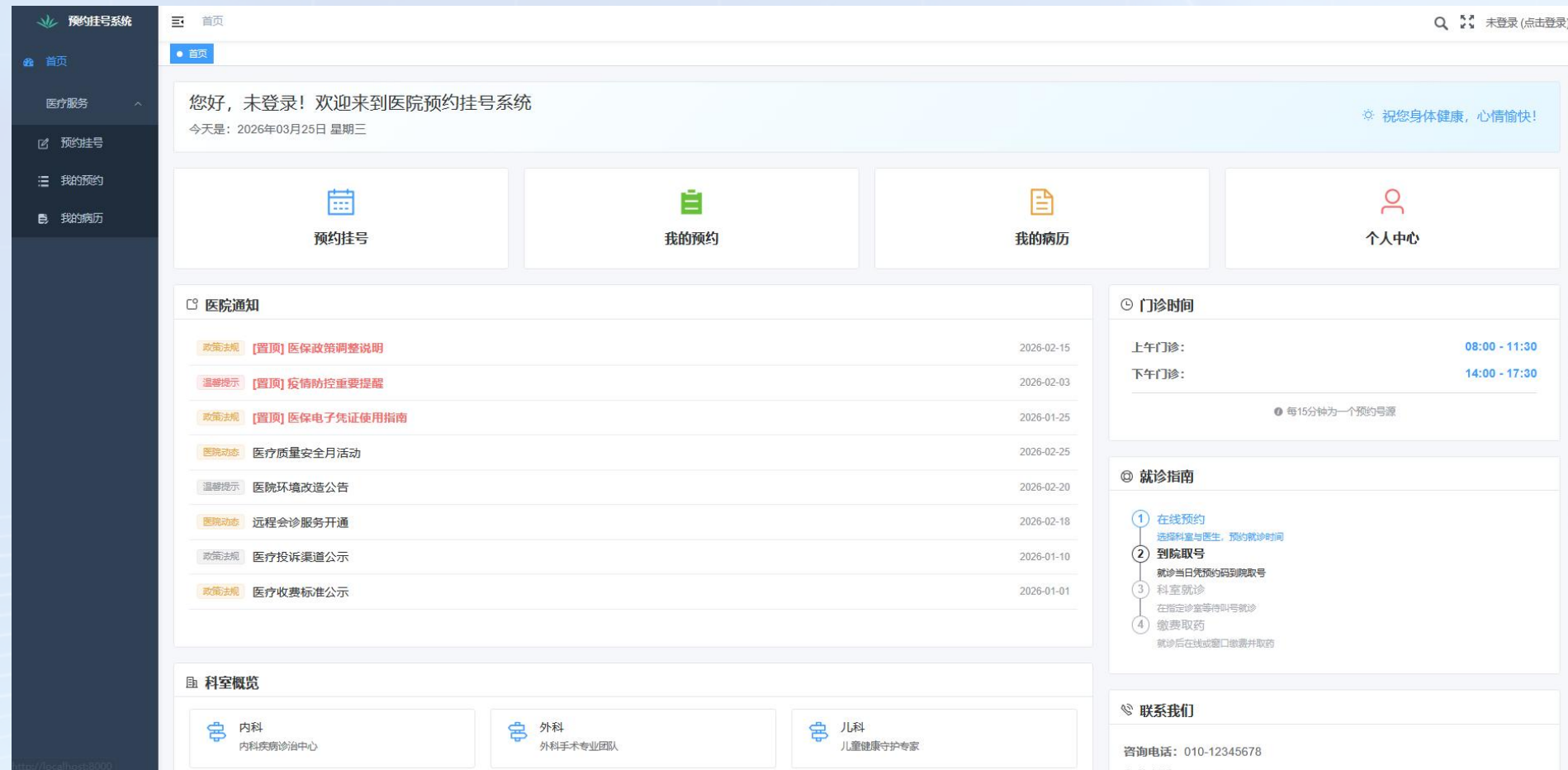
改动目的

以此降低接口暴露风险*；简化普通用户操作路径，提升体验。

The image displays three login form mockups arranged in a collage. The top mockup is titled '管理员登录' (Administrator Login) and features tabs for '管理员' (Administrator), '患者' (Patient), and '医生' (Doctor). It includes input fields for '账号' (Account), '密码' (Password), and '验证码' (Verification Code), along with a '记住密码' (Remember Password) checkbox and a '登录' (Login) button. The bottom-left mockup is titled '患者登录' (Patient Login) and includes input fields for '身份证号/手机号' (ID Card Number/Phone Number), '密码' (Password), and '验证码' (Verification Code), with a '记住密码' (Remember Password) checkbox and a '登录' (Login) button. The bottom-right mockup is titled '管理员登录' (Administrator Login) and includes input fields for 'admin' (username), 'password', and '验证码' (Verification Code), with a '记住密码' (Remember Password) checkbox and a '登录' (Login) button. All forms include a '立即注册 访客登录 >>' (Register Now Visitor Login >>) link at the bottom.

3 系统设计与实现：访客界面展示

访客角色界面展示



功能描述：

访客可查看排班与号源信息，其余详细数据需登录后访问，更贴合实际医院挂号场景

可用功能：

- 医院首页与介绍
- 科室导航与简介
- 医生信息公开查询
- 医院新闻与公告展示
- 号源查看

3 系统设计与实现：用户角色功能与界面展示



患者角色

注册登录 · 号源筛选
预约挂号 · 取消预约
电子病历 · 就诊记录



DOCTOR
PROFESSIONS

医生角色

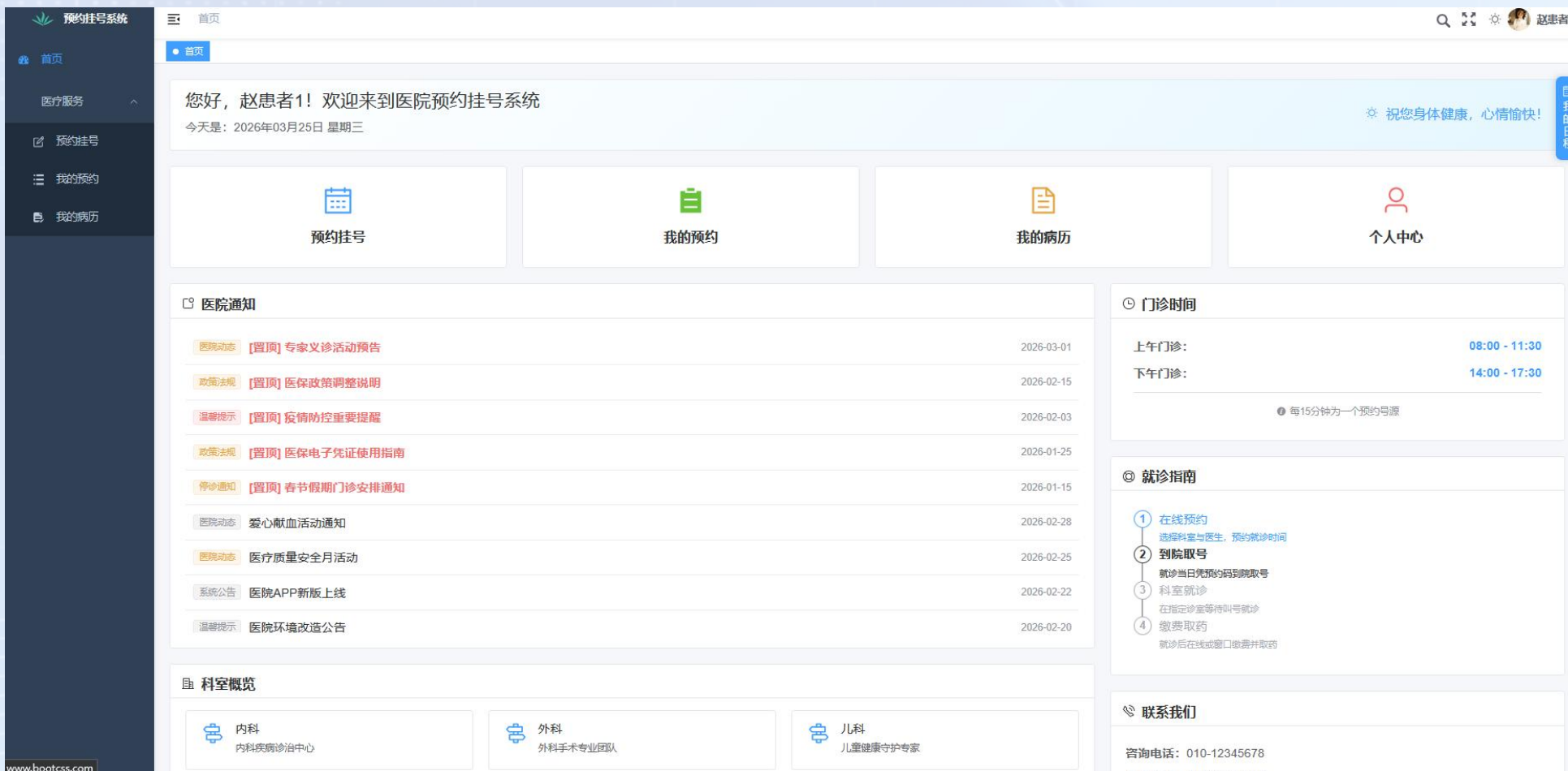
排班管理 · 接诊标记
病历录入 · 病历撰写



管理员角色

用户管理 · 科室调配
号源管理 · 权限管控
数据统计 · 系统配置

3 系统设计与实现：用户角色功能与界面展示



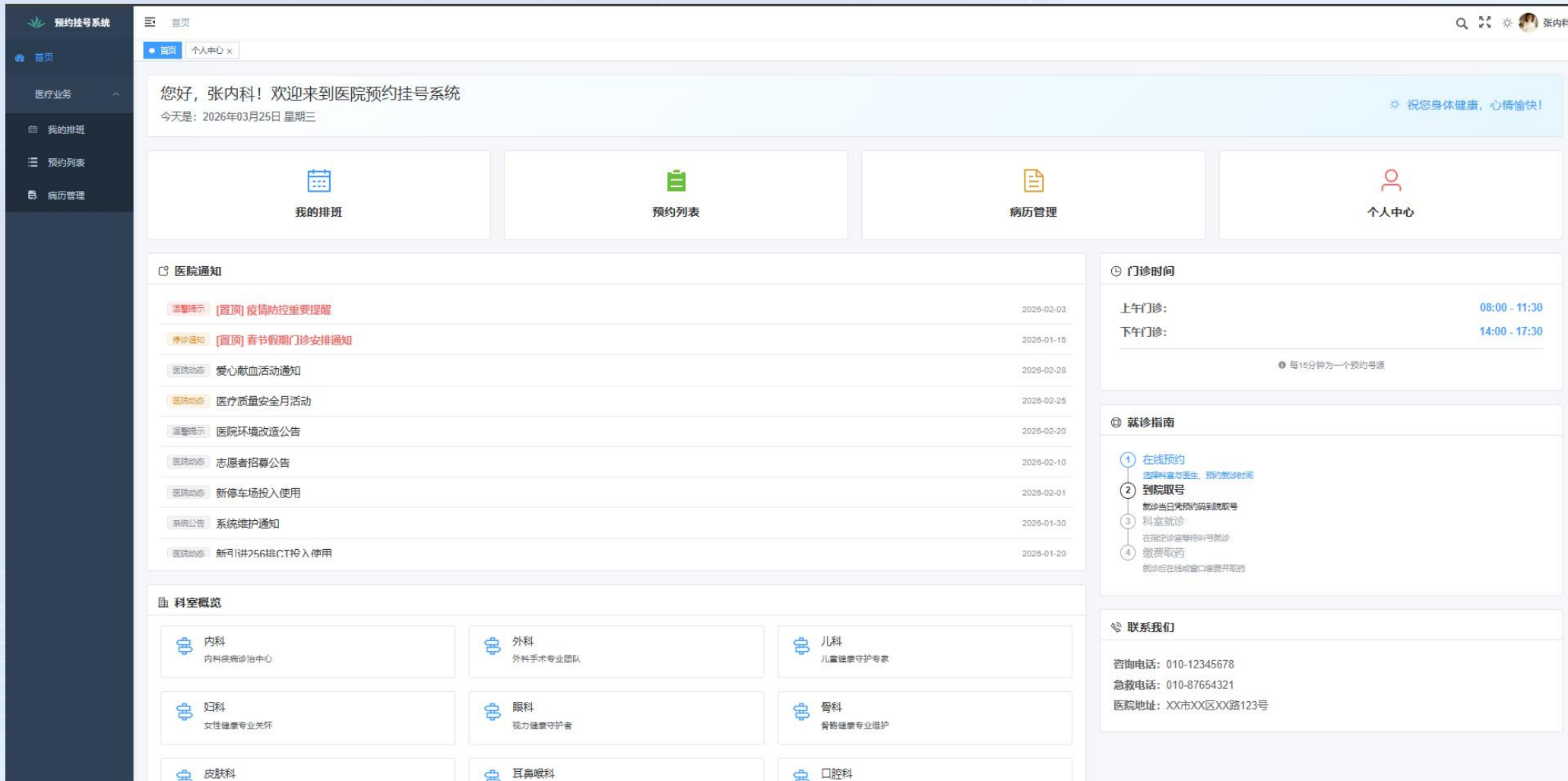
患者角色

注册登录 · 号源筛选
预约挂号 · 取消预约
电子病历 · 就诊记录

功能描述:

患者核心功能为预约挂号: 选择科室可查看该科室医生, 筛选日期后展示当日排班医生, 选择时段即可完成预约

3 系统设计与实现：用户角色功能与界面展示



医生角色

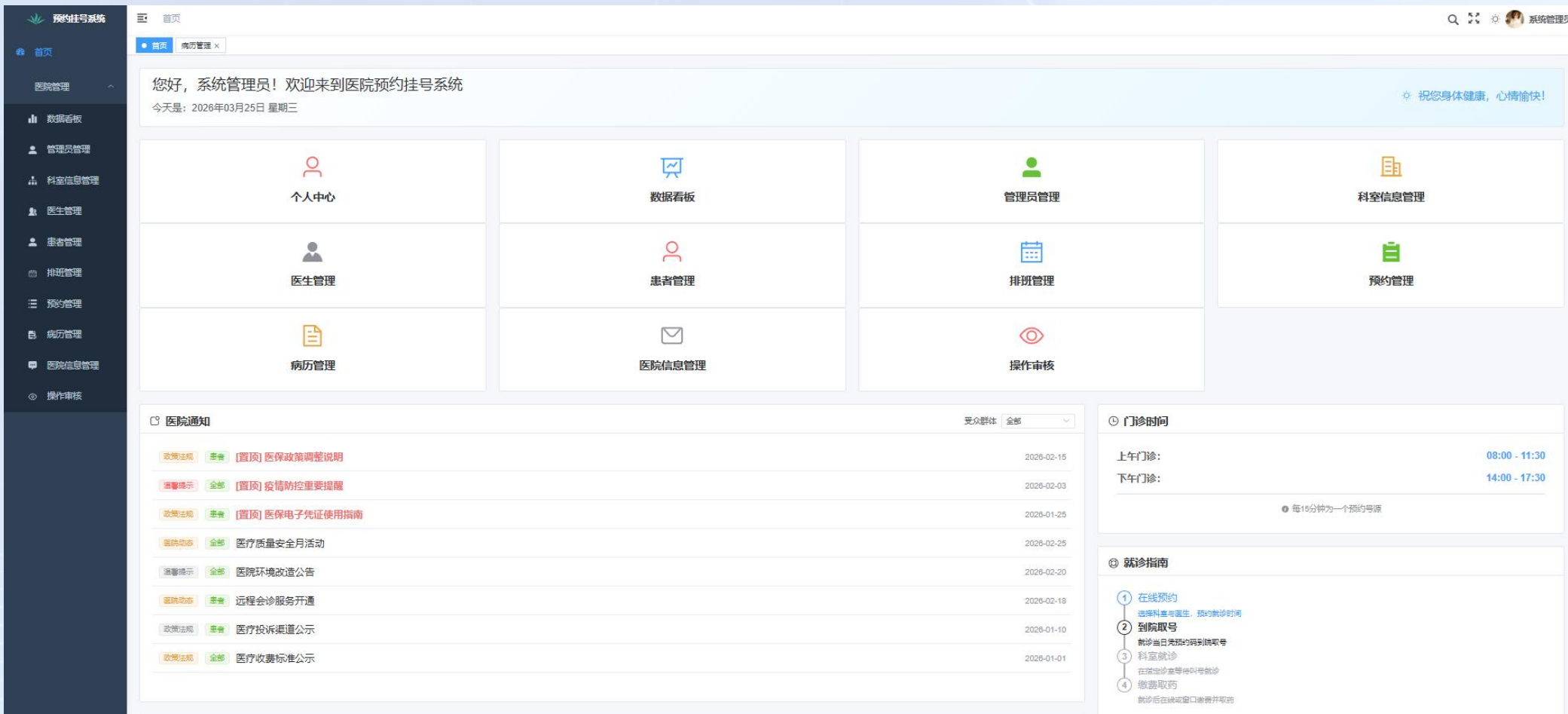
排班管理 · 接诊标记
病历撰写

功能描述：

医生核心功能为就诊与排班：可在排班页设置信息并取消排班，若有患者预约，将同步取消对应预约

就诊从预约列表进入，可按日期筛选预约情况，开始就诊后进入就诊台，查看基础信息、书写病历并查看患者就诊记录

3 系统设计与实现：用户角色功能与界面展示



管理员角色

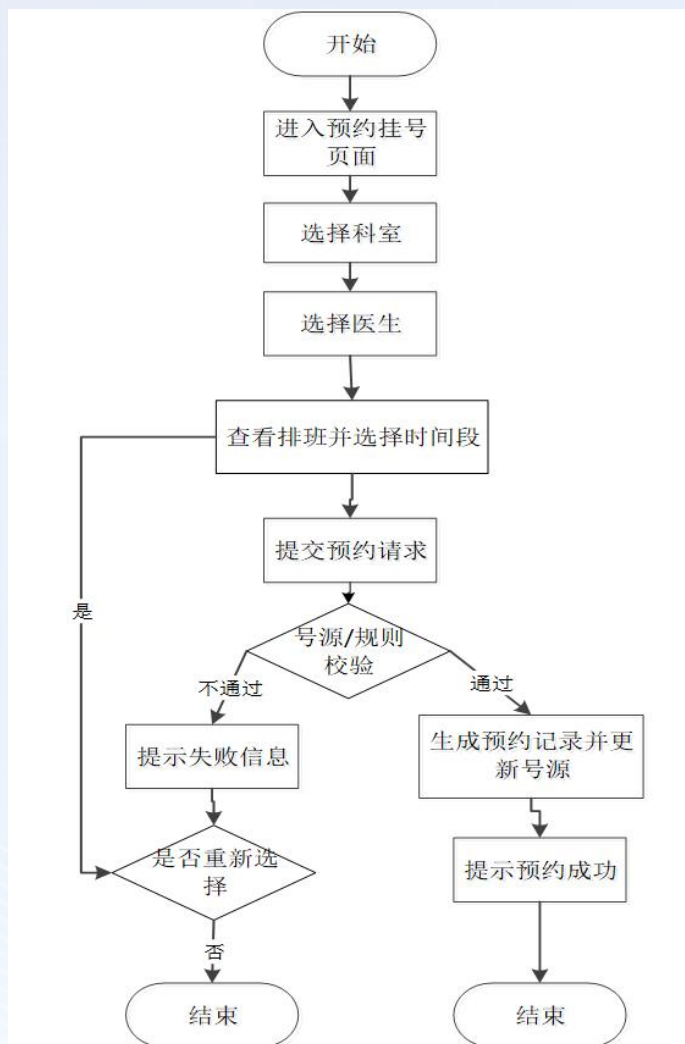
用户管理 · 科室调配
号源管理 · 权限管控
数据统计 · 系统配置

功能描述：

管理员端核心功能为基础信息维护与操作审核，可修改各模块信息，审核患者与医生发起的相关操作

3 系统设计与实现：核心业务流程

01 患者挂号流程



用户登录

身份验证与授权

选择预约

科室/医生/时段

号源校验

库存与资格检查

生成记录

创建预约

号源扣减

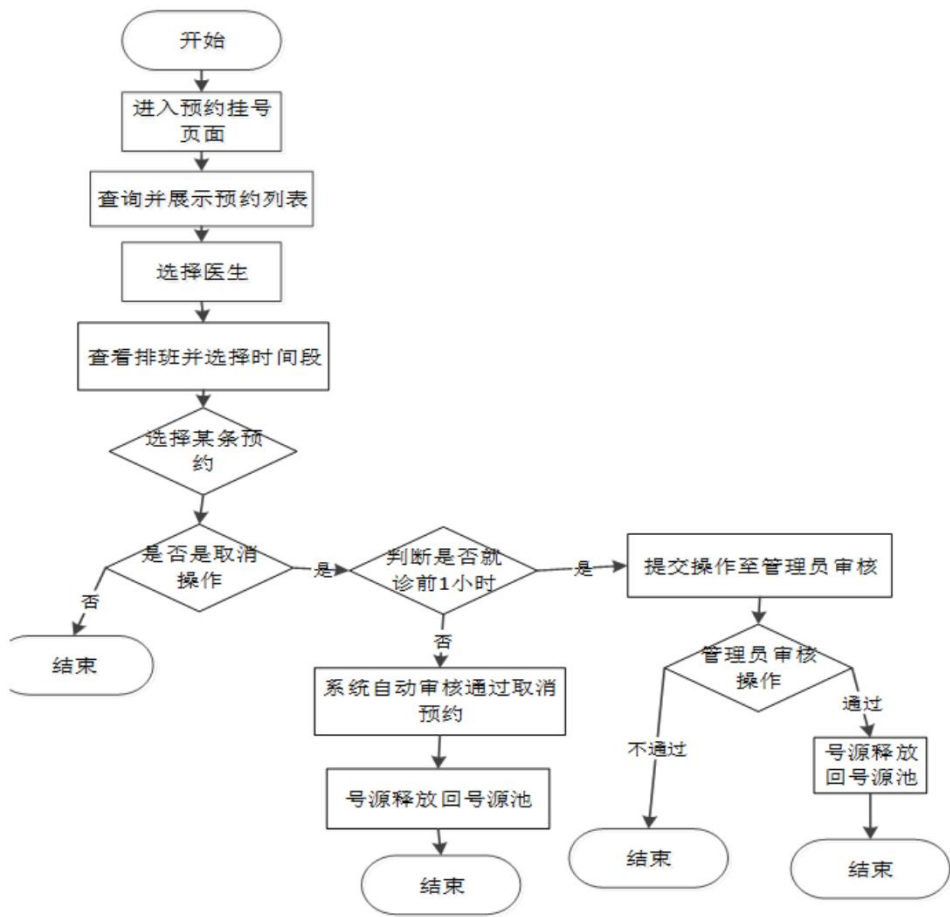
更新剩余库存

流程描述：

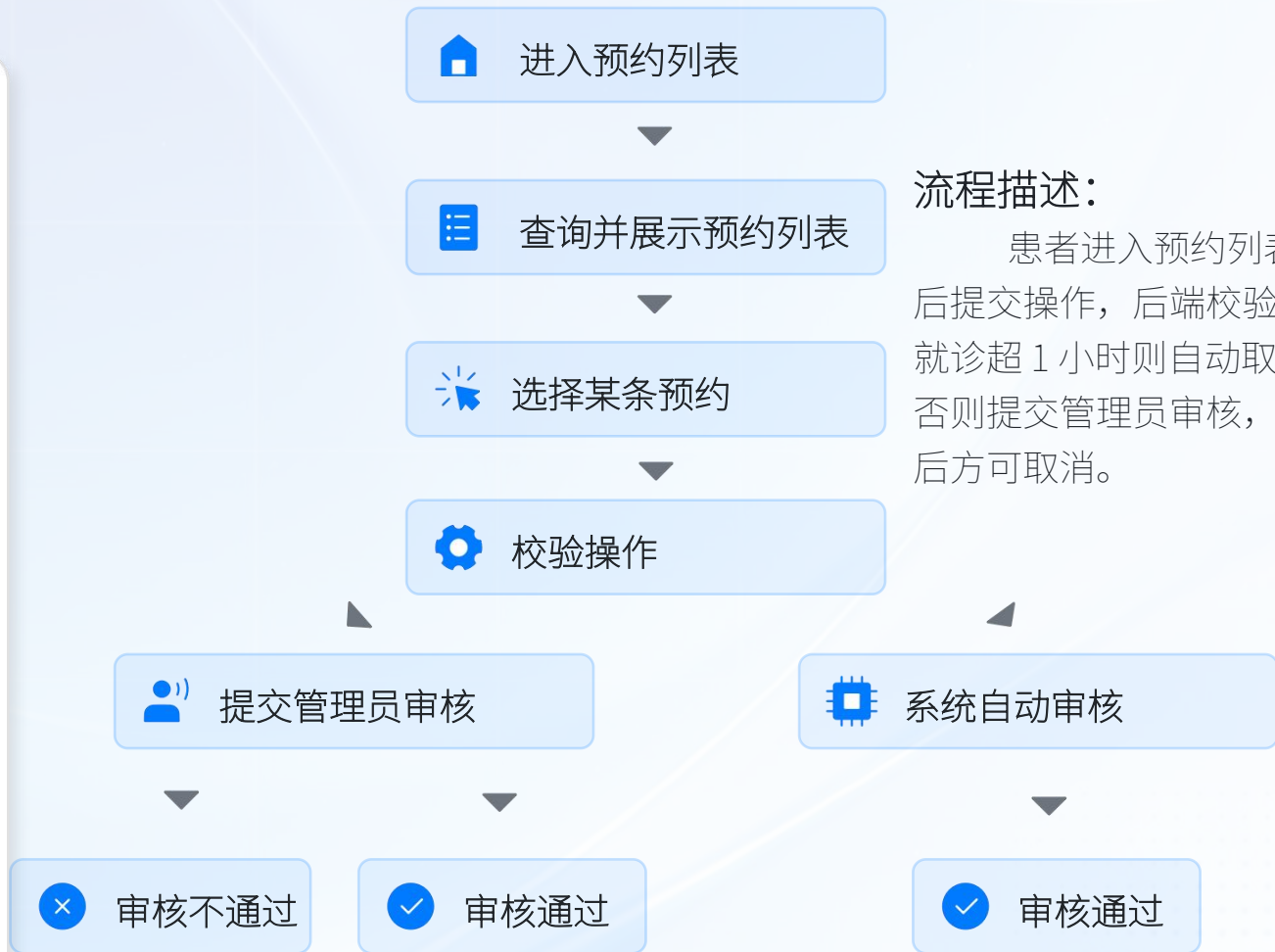
患者登录后查看号源，
预约号源，后端校验号源，
校验成功创建预约并扣减号
源

03 系统设计与实现：核心业务流程

02 患者预约管理流程

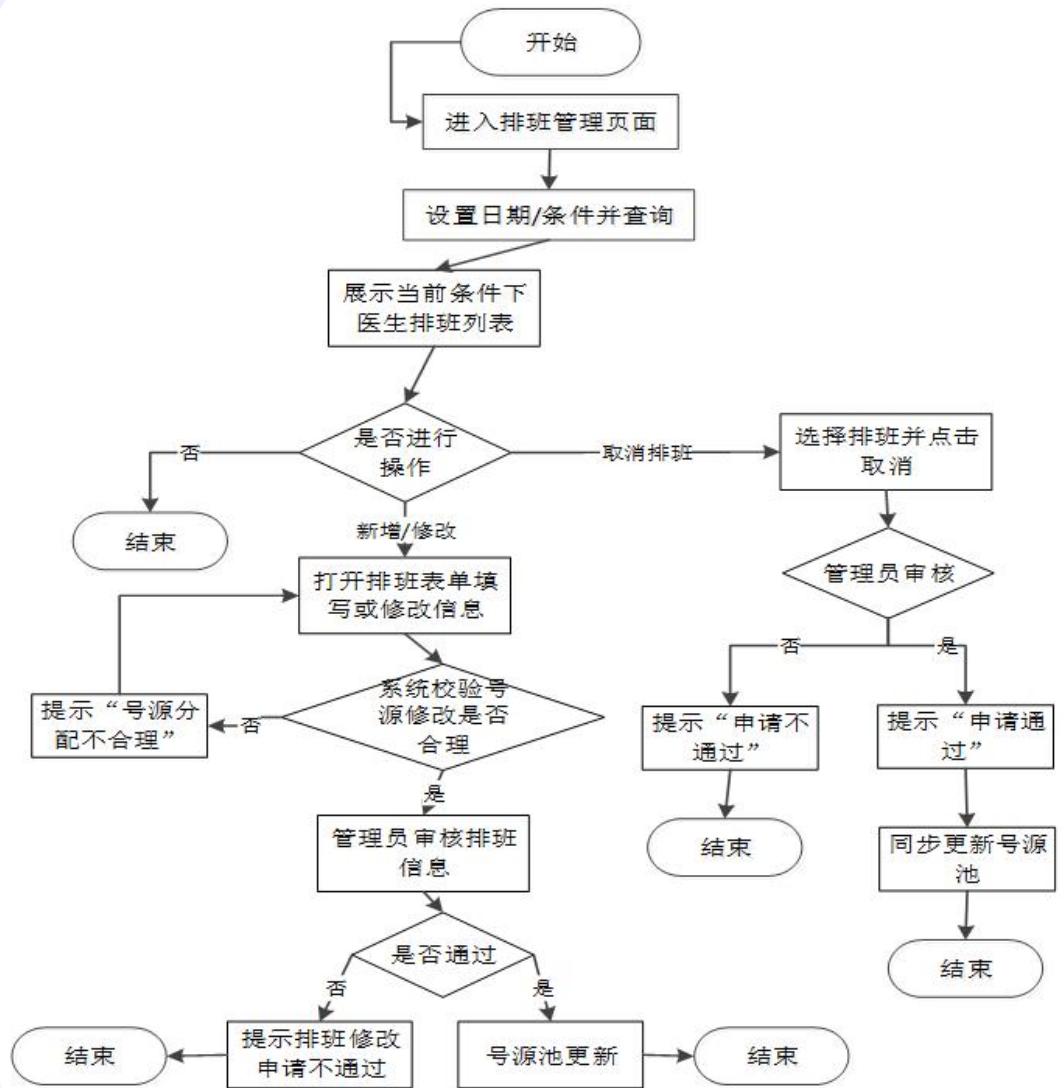


患者操作预约管理流程示意图



03 系统设计与实现：核心业务流程

03 医生排班管理流程



医生排班管理流程示意图



01. 进入排班管理页面

医生登录系统后，进入排班管理功能模块



02. 查询排班列表

设置日期或其他条件，查询并展示当前排班



03. 选择操作类型

选择对排班进行“新增/修改”或“取消”操作



04. 新增/修改排班

填写或修改信息，系统校验号源分配合理性



05. 取消排班

选择需要取消的排班记录并提交申请



06. 管理员审核

对现有排班变动操作均需提交给管理员进行审核



07. 审核通过

系统自动更新号源池，排班生效或取消



08. 审核不通过

审核不通过，排班操作失败，流程结束

流程描述：

医生进入排班列表查询信息后操作，新增或修改时系统校验号源合理性，对已有排班的变动需经管理员审核

★ 3 系统设计与实现：系统实现亮点



Redis缓存优化

缓存排班数据，实现防穿透、防雪崩，大幅提升查询效率。



号源防超卖

通过Redis原子操作和数据库事务，确保高并发下数据一致性。



微服务熔断降级

配置FallbackFactory和sentinel接口，防止服务雪崩，保障核心功能。

★ 3 系统设计与实现：系统实现亮点

```
// 空值缓存过期时间（防止缓存穿透）
private static final long NULL_CACHE_EXPIRE = 60L;
...
Integer availableSlots = redisService.getCacheObject(redisKey);

// 缓存穿透保护：如果缓存中是特殊标记（如-1），说明数据库也不存在
if (availableSlots != null && availableSlots == -1) {
    return null;
}
...
if (schedule != null) {
    ...
} else {
    // 数据库也不存在，缓存空值（-1），短过期
    redisService.setCacheObject(redisKey, -1, NULL_CACHE_EXPIRE, TimeUnit.SECONDS);
}
```

```
// 获取带有随机波动的过期时间，防止缓存雪崩
private long getRandomExpire() {
    return BASE_CACHE_EXPIRE + new Random().nextInt(3600);
}
...
// 缓存未命中，写入缓存（防止缓存雪崩：设置随机过期时间）
redisService.setCacheObject(redisKey, schedule.getAvailableSlots(), getRandomExpire(), TimeUnit.SECONDS);
```

优化缓存穿透

执行流程：

1. 先从Redis查找热点key
hospital:schedule:slots:{id} 中是否存在当前排班ID。
2. 命中-1：如果Redis中命中的值为 -1，说明该ID不存在，直接返回 null，不再查询数据库。
3. Redis未命中：如果Redis中没有该key，则去查询数据库。
4. 数据库未命中：如果数据库也查询不到该排班ID，则将 -1 写入Redis，并设置一个较短的过期时间（NULL_CACHE_EXPIRE = 60s）。

流程总结：查询无排班数据时缓存key设置 - 1 并设短过期，拦截无效请求，减轻 DB 压力。



Redis缓存优化

缓存热点数据，实现防穿透、防雪崩。

优化缓存雪崩

执行流程：

1. 统一设置基础过期时间，引入随机过期偏值
2. 所有写入缓存的步骤都带入随机TTL值

流程总结：给缓存 key 设置基础 + 随机过期时间，避免批量失效

★ 3 系统设计与实现：系统实现亮点

```
// 1) DECR: 先做总号源原子扣减
remaining = redisService.redisTemplate.opsForValue().decrement(redisKey);
if (remaining != null && remaining < 0) {
    // 扣减后小于0, 说明无号源, 立即回滚并拒绝
    redisService.redisTemplate.opsForValue().increment(redisKey);
    throw new ServiceException("对不起, 该时段号源已满");
}

// 2) SETNX: 对“排班+时段”做互斥占位
if (appointment.getAppointmentTime() != null && !appointment.getAppointmentTime().trim().isEmpty()) {
    slotLockKey = getAppointmentSlotLockKey(scheduleId, appointment.getAppointmentTime());
    Boolean lockResult = redisService.redisTemplate.opsForValue().setIfAbsent(
        slotLockKey,
        String.valueOf(appointment.getPatientId()),
        24L,
        java.util.concurrent.TimeUnit.HOURS
    );

    if (!Boolean.TRUE.equals(lockResult)) {
        // 时段已被占, 回滚刚才的库存扣减
        redisService.redisTemplate.opsForValue().increment(redisKey);
        throw new ServiceException("该具体时段已被预约, 请选择其他时段");
    }
}
```

号源防超卖实现原理

1. 核心实现逻辑

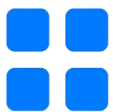
对 scheduleId + appointmentTime 做分布式互斥锁：
使用redis的原子操作指令DECR加上设置分布式锁SETNX
控制号源不会超卖以及确保号源的唯一性

Redis DECR 先扣总号源库存
若 该时段总号源<0 立即 INCR 回滚并失败
再 SETNX 抢“排班+时段”锁
抢锁失败再 INCR 回滚并失败
抢锁成功才继续后续流程（远程同步/落库）
这个顺序能保证两点：
不会把总库存扣成负数（有回滚）
同一具体时段不会被并发重复占用（有 SETNX）

2. 最终实现

DECR 控号源总量，SETNX 控预约的唯一性（唯一排班ID：就诊时段），二者组合才能真正防止高并发下的错卖和超卖。

4 系统测试与结果



功能测试

登录权限、预约挂号、数据维护等全流程功能测试均合格，核心业务流程闭环完整。



安全测试

mapper.xml中不使用sql拼接语句，使用参数绑定和字段过滤，拦截SQL注入攻击



并发测试

模拟200人同时抢1个号源，系统表现稳定，无超卖现象，数据一致性保持良好。



可用性测试

多终端界面适配良好，操作流程符合用户习惯，错误反馈提示清晰易懂。

4 系统测试与结果



Jmeter做压力测试

使用JMeter进行接口参数配置，添加两百个患者账号用于测试，对一个号源进行并发抢号



并发测试

模拟200人同时抢1个号源，系统表现稳定，无超卖现象，数据一致性保持良好，多次测试过后得出：瞬时峰值QPS可达200。

host	localhost
port	8080
scheduled	1722
appointmentTime	09:00:00

预约ID	患者ID	预约ID	状态	预约时间	预约时段 (如08:00:00)	是否	受理医生
1280	198	1722	待就诊	2026-03-25 20	09:00:00	0	(Null)
1263	1	1720	已过期	2026-03-24 19	10:30:00	0	(Null)

实时监控

关键字

升序

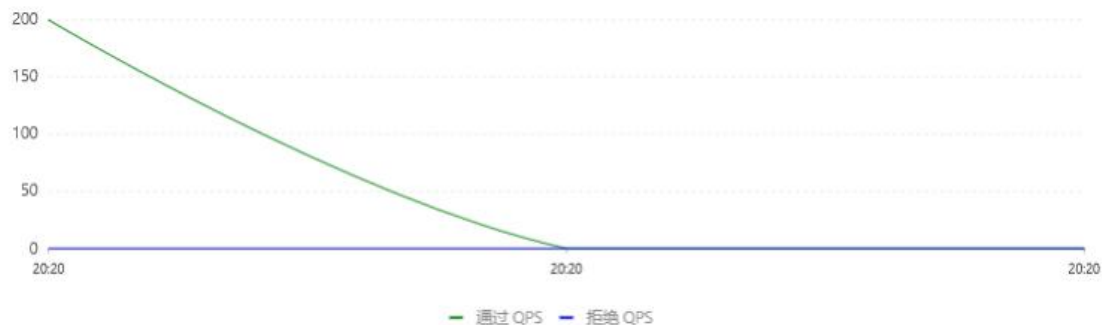
线程属性

线程数: 200

Ramp-Up时间 (秒): 1

循环次数 ☐ 永远 1

/appointment/create



时间	通过 QPS	拒绝QPS	响应时间 (ms)
20:20:44	0.0	0.0	1,905.0
20:20:43	0.0	0.0	588.0
20:20:42	200.0	0.0	248.0
-	-	-	-
-	-	-	-
-	-	-	-

5 论文总结与展望：项目优点与不足

✓ 研究亮点与优势

- 🏢 业务贴合实际：完整还原医院挂号流程，实用性强。
- 📱 技术落地性强：各项技术均按照实际医院挂号逻辑。
- 📊 图表支撑充分：逻辑推导清晰，论证过程详实。
- 🛡️ 测试数据量化：通过多维度测试数据证明系统稳定性。

💡 现存不足与改进方向

- 🗄️ 数据库冗余：部分字段设计冗余，需要优化数据一致性。
- 💻 系统稳定性有待加强：加入MQ消息队列、启用集群保证系统的高可用。
- 🔧 功能待扩展：缺少智能提醒、医保支付等增值功能模块，集成多种通信协议保障数据传输完整性。

总结：立足实际业务实现技术落地，后续将针对数据冗余与模块化不足进行专项优化。

5 论文总结与展望：总结与展望



项目总结

本项目成功设计并实现了一个基于微服务架构的医疗预约挂号系统，满足了基本的业务需求，达到了预期目标。

未来展望



数据库优化

优化数据库设计，实现读写分离，消除数据冗余，提升查询效率。



代码组件化

对核心代码进行组件化封装，降低耦合度，提升系统可维护性。



数据可视化

深化数据可视化功能，优化多端适配体验，提升数据决策价值。



新增智能功能

计划新增智能提醒（短信，邮箱）、医保支付对接、AI辅助分诊等功能，增强智能化。

致 谢

感谢各位老师的聆听！

谢谢！

期待您的提问与建议